Assignment 6B -- Path B

Build the Recipe

Lectern teaching-demand pipeline, conductor-ready: config, verified data, recipe, audit.

Nik Bear Brown

INFO 7375 Branding and AI -- Fall 2026

Due October 23, 2026

Worked example -- drafted 2026-10-11. Gru /snickerdoodle confirmation is TODO (see Part 4).

Overview

Due: October 23, 2026, 11:59 PM · Points: 100
Canvas submissions: LastName_FirstName_A5B_BuildTheRecipe.pdf +
brand_config.json + recipe_[LastName].md

Status (2026-10-11)

Piece	State
`brand_config.json`	[x] Valid JSON, all fields from the Conductor Brief, judgment types named
`data/verified/`	[x] 3 sources: greenhouse (53), ashby (34), smartrecruiters (10) + README
`recipe_brown.md`	[x] Four layers, typed phase gates, typed TODOs, standardization appendix
Part 4 /snickerdoodle	[x] Manual audit documented; **Gru `/claude` confirmation needs Bear**
PDF	[x] `Brown_Nik_A5B_BuildTheRecipe.pdf` (with appendices)

Files

- `brand_config.json` -- machine-readable Conductor Brief (submit separately).
- `recipe_brown.md` -- the conductor's recipe (submit separately).
- `data/verified/{greenhouse,ashby,smartrecruiters}/` -- verified records + README.
- `part-4-snickerdoodle.md` -- pipeline description, audit TODOs, resolutions.
- `make_pdf.py` -- PDF build script.

Honest limits (what needs Bear)

1. **SmartRecruiters is thin** -- 10 verified records vs the 25-record bar.
   TODO: DATA SOURCE in the recipe (wire more boards or accept two sources).
2. **Gru steps** -- `/snickerdoodle` and `/claude` need Gru; the audit here is
   manual and labeled as such. No confirmation is fabricated.
3. **Two TODOs need time** -- trusted-run thresholds want two more weekly runs.
4. Nothing here invents data: every verified record traces to the real
   2026-09-26 run; rejections are documented, not hidden.

Part 1 -- brand_config.json (25 pts)


```
{
 "conductor_note": "Boards change between runs \u2014 judge fetch counts against the source-reported tota
l, never against last week's number alone; and a Monday run will read thinner than a midweek one, which i
s calendar, not signal.",
 "data_sources": [
  {
   "local_path": "data/verified/greenhouse/",
   "name": "Greenhouse public boards API",
   "quality_gate": "Every record carries title, URL, date, location, department; duplicates collapsed on
source + native ID; field positions asserted per board so a layout change fails loudly instead of writing
 blank titles.",
   "record_type": "One job posting (title, URL, date, location, department, ad text) from a Greenhouse-ho
sted company board.",
   "update_freq": "Weekly; the 2026-09-26 run fetched the majority of 3,446 postings across 18 boards."
  },
  {
   "local_path": "data/verified/ashby/",
   "name": "Ashby posting API",
   "quality_gate": "Same record standard as Greenhouse; raw responses saved untouched and dated.",
   "record_type": "One job posting from an Ashby-hosted board (AI labs: OpenAI, Anthropic, Writer, Notion
, and others).",
   "update_freq": "Weekly, same run cadence as Greenhouse."
  },
  {
   "local_path": "data/verified/smartrecruiters/",
   "name": "SmartRecruiters postings API",
   "quality_gate": "Same record standard; the paging loop is verified to emit no duplicates across pages.
 NOTE: only 10 verified records in the 2026-09-26 run \u2014 below the 25-record bar; see recipe TODO: DA
TA SOURCE.",
   "record_type": "One job posting from a SmartRecruiters-hosted board (Canva); paged 100 at a time with
one extra call per posting for the description.",
   "update_freq": "Weekly, same run cadence."
  }
 ],
 "one_thing": "Lectern is the only job-board reader that turns listings into checkable evidence of a comp
any's interest in teaching, for the independent education consultant, without asking them to pretend to b
e a job seeker.",
 "phase_gates": [
  {
   "bad_looks_like": "A whole ATS returns nothing, or a board that had 248 postings returns 12.",
   "good_looks_like": "Every source returned its expected board count; raw responses saved untouched, dat
ed, per source; fetch counts recorded next to the source-reported totals. (A Figma fetch of 151 against a
 160-posting snapshot is expected drift; a fetch of 12 is not.)",
   "judgment_type": "IJ",
   "response": "Log and continue \u2014 the failure is recorded as unavailable and the other sources keep
 going. Flag for review if an entire ATS is dark across two runs.",
   "step": "Data ingestion"
  },
  {
   "bad_looks_like": "Field assertions raise (the board changed its layout), or a run lands under 90% of
the reported total.",
   "good_looks_like": "100% of records carry title, URL, date, location, department; duplicates collapsed
 on source + native ID (0 remaining); field assertions passed. A run is trusted only at >=90% of the sour
ce-reported total and >=70% of the previous trusted run.",
   "judgment_type": "IJ",
   "response": "Hard stop for that source's downstream \u2014 nothing blank is ever written, and an untru
sted run freezes the closed-job logic (it may update 'seen' timestamps, but it closes nothing).",
   "step": "Data validation (GIGO gate)"
  },
  {
   "bad_looks_like": "A rule keeps postings at ~1-in-10 precision (as the first teaching-materials rule d
```

```
id), or a demand claim appears with no raw link behind it.",
   "good_looks_like": "Every kept posting names the rule that fired and the words that matched; every rej
ected posting names its reason; the demand ranking links each company claim to its signal roles; the new/
gone diff compares native IDs across two trusted runs. Trust test: pick any claim at random \u2014 you ca
n reach the raw posting in two clicks.",
   "judgment_type": "TO",
   "response": "Flag for review, then fix and re-audit. An unlinked claim is a bug, not a judgment call:
hard stop until it is traced or removed.",
   "step": "Intelligence layer"
  },
  {
   "bad_looks_like": "Numbers that cannot be recomputed from committed files, or limits hidden \u2014 an
unreadable company presented as 'no demand.' (Derived from Part 4's good-looks-like; Part 5 defines no St
ep 4 failure mode.)",
   "good_looks_like": "A non-technical reader can answer 'which companies, what roles, what's new this mo
nth' from page one; every number is recomputable from the committed files; the report states its limits (
unreadable companies, unverified totals) instead of hiding them.",
   "judgment_type": "TO",
   "response": "Flag for review \u2014 a human reads the report before it ships.",
   "step": "Final report"
  }
 ],
 "primary_user": {
  "role": "Independent AI-education consultant \u2014 typically a working professor or former educator se
lling teaching expertise to software vendors.",
  "situation": "They want an education-advocate role (flexible, part-time, or consulting) with a company
whose tools they already teach. That role is almost never posted, so the real job is figuring out which c
ompanies are worth approaching, and when.",
  "workaround": "Scrolling careers pages by hand (an hour a week, first thing to get skipped), LinkedIn j
ob alerts on titles like 'advocate,' and conference chatter."
 },
 "tool_summary": "Lectern helps an independent AI-education consultant find which companies selling AI to
 universities are showing real interest in how their tools are used in teaching, so they can pitch a role
 that hasn't been posted yet."
}
```

Part 2 -- Verified data folder (20 pts)

Verified inputs for the Lectern recipe (Assignment 5B, Path B). Every record below passed the GIGO gate from Conductor Brief Part 3. No raw or unvalidated records live here.

Folders

- `greenhouse/` -- **53 records** | 2026-09-26 | Kept postings from Greenhouse-hosted boards (design-tool companies). File: `verified-2026-09-26.json`.
- `ashby/` -- **34 records** | 2026-09-26 | Kept postings from Ashby-hosted boards (AI labs: OpenAI, Anthropic, Writer, Notion, others). File: `verified-2026-09-26.json`.
- `smartrecruiters/` -- **10 records** | 2026-09-26 | Kept postings from SmartRecruiters-hosted boards (Canva). File: `verified-2026-09-26.json`. **Below the 25-record bar** -- see below.

Quality gate applied (all three folders)

A record is admitted only if it has all five fields -- title, URL, date, location, department -- is deduplicated on source + native ID, and was kept by the teaching-interest filter with its rule trace intact (kept_because + matched words on every record). Source: the 2026-09-26 run (3,446 postings fetched, 97 kept, 0 duplicates -- see build-your-data-pipeline/ for the run files and both audits).

Records rejected and why

- **Greenhouse:** the run fetched the majority of 3,446 postings; the filter kept 53. The rest failed the teaching-interest rules (no role/title/topic match) -- rejections are logged with reasons in the run files, not here.
- **Ashby:** same -- 34 kept of the fetched postings; the rest rejected by rule.
- **SmartRecruiters:** 10 kept. The Canva board fetched 248 postings; only 10 matched the teaching-interest filter. This is a thin source: it clears the quality gate but does not meet the 25-record minimum for a verified source folder. Recorded as TODO: DATA SOURCE in the recipe -- either more SmartRecruiters boards get wired up, or the recipe runs on two sources with the gap labeled.

What this folder does not cover (honest limits, from Brief Part 3)

Adobe (Workday) and Framer (hand-built pages) have no public feed -- absence is a measurement gap, never "no demand." Amazon's hits: 10000 is trusted but unverified as a cap. Big-four runs exist only in uncommitted logs.

Part 3 -- Recipe file (25 pts)

Version: 1.0 | Last run: 2026-09-26 | Run frequency: Weekly (on demand for a pitch push)

--- EXECUTIVE SUMMARY ---

This recipe runs Lectern's weekly demand scan: it fetches job postings from three public ATS feeds (Greenhouse, Ashby, SmartRecruiters), validates every record against the GIGO gate, keeps the postings that are evidence of teaching interest (with the rule that fired on each), and produces a company demand report the conductor's user -- the independent AI-education consultant -- reads to decide who is worth approaching about unposted work. The conductor is whoever runs it (Claude Code, a colleague, future-you); a successful run produces logs/reports/demand-YYYY-MM-DD.md with every claim traceable to a raw posting in two clicks, plus a reject log with reasons.

--- LAYER 1: EXTERNAL SOURCES -> data/raw/ ---

Step 1.1 Greenhouse public boards API
  Action: One keyless JSON call per watched board (18 boards in the 2026-09-26 run); save each raw response untouched, dated, per board.
  Output: data/raw/greenhouse/YYYY-MM-DD/<board>.json
  TODO: RESOLVED -- collector exists (build-your-data-pipeline/collect.py); field positions asserted per board so layout changes fail loudly.

Step 1.2 Ashby posting API
  Action: One keyless JSON call per board (board names URL-encoded); same raw-save rule as 1.1.
  Output: data/raw/ashby/YYYY-MM-DD/<board>.json
  TODO: RESOLVED -- same collector, same assertions.

Step 1.3 SmartRecruiters postings API
  Action: Paged fetch (100/page), then one extra call per posting for the description (Canva: 248 postings, 251 requests, ~3.5 min).
  Output: data/raw/smartrecruiters/YYYY-MM-DD/<board>.json
  TODO: TODO: DATA SOURCE -- only one SmartRecruiters board (Canva) is wired up; 10 verified records in the last run, below the 25-record bar. Resolve by wiring more boards or accepting a two-source recipe.

PHASE GATE 1 | Judgment type: IJ (Ingest Judgment)
  Good looks like: Every source returned its expected board count; raw responses saved untouched, dated, per source; fetch counts recorded next to the source-reported totals. (Figma 151 vs a 160-posting snapshot is expected drift; 12 is not.)
  Bad looks like: A whole ATS returns nothing, or a board that had 248 postings returns 12.
  Response: Log and continue -- record the source as unavailable and let the other sources finish. Flag for review if an entire ATS is dark across two consecutive runs.

--- LAYER 2: data/raw/ -> data/verified/ ---

Step 2.1 Record validation (GIGO gate)
  Action: Assert every record carries title, URL, date, location,
           department; collapse duplicates on source + native ID; run the
           field-position assertions per board.
  Output: data/verified/<source>/verified-YYYY-MM-DD.json
  Rejects: data/raw/rejected/<source>/ with the failed assertion logged per
           record. Nothing blank is ever written downstream.
  TODO: TODO: DEFINE -- the trusted-run thresholds (>=90% of source-reported
           total, >=70% of previous trusted run) are set from the 2026-09-26
           run; confirm they still fit once two more weekly runs exist.

Step 2.2 Teaching-interest filter
  Action: Apply the rule set (role words in title, topic words in text,
           materials rule as currently tuned); every kept record stores
           kept_because + matched words; every rejected record stores its
           reason.
  Output: data/verified/<source>/verified-YYYY-MM-DD.json (kept records only)
  TODO: RESOLVED -- rules + the audit loop that cut nine phrase groups after
           the materials rule failed at ~1-in-10 (now 5 kept, 4 right).

PHASE GATE 2 | Judgment type: IJ (Ingest Judgment)
  Good looks like: 100% of records carry the five fields; 0 duplicates remain;
    field assertions passed; the run clears the trusted-run thresholds.
  Bad looks like: Field assertions raise (board layout changed), or the run
    lands under 90% of the reported total.
  Response: Hard stop for that source's downstream. An untrusted run
    freezes the closed-job logic -- it may update "seen" timestamps, but it
    closes nothing. This is the most important gate: a bad run must not
    declare jobs dead.

--- LAYER 3: TOOL / AI PROCESSING ---

Step 3.1 Demand ranking
  Action: Aggregate kept postings per company; rank by signal-role count;
           diff native IDs against the previous trusted run for new/gone.
  Model: No LLM in the loop -- deterministic rules + audit loop. (Deliberate:
           the differentiator is verifiability, and a model in this step would
           make claims uncheckable.)
  Output: logs/reports/demand-YYYY-MM-DD.md (draft)
  TODO: TODO: DEV -- the new/gone diff currently runs by hand from the run
           files; script it so the conductor gets it automatically.

Step 3.2 Claim trace check
  Action: Automated pass: every company claim in the draft must resolve to >=1
           kept record with a live URL; any unlinked claim is removed or
           flagged before the gate.
  Output: Same draft, trace-checked.

TODO: RESOLVED -- the two-click trust test (brief Part 4, Step 3).

PHASE GATE 3 | Judgment type: TO (Tool Output Review)
Good looks like: Every kept posting names its rule and matched words; every rejected posting names its reason; the ranking links each claim to signal roles; the diff compares native IDs across two trusted runs.
Bad looks like: A rule keeps postings at ~1-in-10 precision, or a claim appears with no raw link behind it.
Response: Flag for review, then fix and re-audit. An unlinked claim is a hard stop until traced or removed.

--- LAYER 4: logs/reports/ -> USER ---

Step 4.1 Report delivery
Action: Render the final demand report (companies, signal roles, what's new this month, stated limits); commit it under logs/reports/.
Output: logs/reports/demand-YYYY-MM-DD.md
TODO: TODO: REPORT FIELD -- the "what's new this month" section needs a human label for the top-line takeaway before it is user-facing (e.g. "Figma is staffing education faster than any watched company").

Step 4.2 Run log
Action: Append fetch counts, keep/reject counts, gate outcomes, and any flags to the run log.
Output: logs/reports/run-log.md (append)

PHASE GATE 4 | Judgment type: TO (Tool Output Review)
Good looks like: A non-technical reader answers "which companies, what roles, what's new" from page one; every number recomputable from committed files; limits stated, not hidden.
Bad looks like: Numbers that can't be recomputed, or an unreadable company presented as "no demand."
Response: Flag for review -- a human reads the report before it ships.

--- CONDUCTOR NOTE ---

Watch the calendar, not just the counts. Boards are quieter on weekends and Mondays -- a thin Monday run is usually calendar, not signal; compare against the source-reported total for that day, never against last week's number alone. And when a company you expected to see goes silent, check whether the board moved (Greenhouse -> Ashby migrations happen) before reading it as lost interest: absence from the data is a measurement gap until proven otherwise. The recipe cannot tell the difference -- you can, by opening the board.

--- OPEN TODOs ---
TODO: DATA SOURCE Wire up more SmartRecruiters boards (or accept a two-source recipe) -- 10 verified records is below the bar.
TODO: DEFINE Confirm the trusted-run thresholds (>=90% / >=70%) after two more weekly runs.

TODO: DEV Script the new/gone native-ID diff (currently by hand).
TODO: APPROVE Human approves the final demand report before it ships (Phase Gate 4) -- the conductor stops here every run.
TODO: REPORT FIELD Human writes the top-line "what's new" takeaway label.

--- APPENDIX: STANDARDIZATION (per recipes/cowork_recipe_standardize.md) ---

Phase-gate test commands

1. Gate 1 -- Ingest complete: every source folder under data/raw/<source>/ holds one dated JSON per watched board; fetch counts logged next to source-reported totals.
   Test: ls data/raw//[0-9]/ | wc -l equals the watched-board count (18), and no board file is empty.
   Human capacity: [IJ].
2. Gate 2 -- GIGO gate: all five fields present on 100% of records; zero duplicates on source+native ID; trusted-run thresholds cleared.
   Test: python3 -c "import json,glob; rs=[r for f in glob.glob('data/verified//.json') for r in json.load(open(f))['records']]; assert all(all(k in r for k in ('title','url','date_posted','location','department')) for r in rs); assert len({(r['source'],r['source_id']) for r in rs})==len(rs); print(len(rs),'records, 0 dupes')"
   Human capacity: [IJ].
3. Gate 3 -- Intelligence traceable: every kept record carries kept_because + matched_words; every company claim in the draft resolves to a kept record.
   Test: grep the draft report for](http -- claim count equals link count; any claim without a link fails the gate.
   Human capacity: [TO].
4. Gate 4 -- Report shippable: page one answers which companies / what roles / what's new; every number recomputable from committed files.
   Test: pick any number in the report at random and recompute it from the committed run files in under five minutes; if you cannot, the gate fails.
   Human capacity: [TO].

Output contract

Agent output

File: logs/lectern-[DATE].json
Fields: workflow, run_id, mode, steps_completed, records_seen, rejects, duplicates, flags, stop_conditions, generated_at, plus per-source fetch_counts and the trusted_run verdict (true/false with the two threshold comparisons).

Human report

File: logs/reports/demand-[DATE].md
Reader: the independent AI-education consultant (Part 2 user).
Decision enabled: which companies to approach about unposted education work, and with what timing evidence.
Sections: ranked company list; signal roles per company (title, link, rule

fired); new/gone vs previous trusted run; stated limits.

Stop conditions

- Stop if any source's field assertions raise (board layout changed) -- the conductor fixes the assertions before that source continues; other sources are unaffected.
- Stop if the run falls below the trusted-run thresholds -- the closed-job logic freezes; nothing is declared dead on an untrusted run.
- Stop if a demand claim has no raw link -- remove or trace it before the report ships.
- Stop at Phase Gate 4 every run -- the report never ships without human approval.

Snickerdoodle

Full dialogic run: snickerdoodle run lectern --mode dialogic
Sample mode (no live network calls, no writes):
snickerdoodle run lectern --mode dialogic --sample
Step: snickerdoodle run lectern --step ingest-greenhouse --sample --no-write
Gate: snickerdoodle gate lectern --gate 2 --decision approve --note "..."

Part 4 -- The /snickerdoodle run (10 pts)

Status: documented as a manual audit -- Gru step needs Bear

/snickerdoodle is a Gru command and the /claude confirmation happens in Gru; neither is available in this environment. What follows is the honest equivalent: the pipeline description that would be fed to /snickerdoodle, plus a manual audit of the recipe against the standardization rules (madison/recipes/cowork_recipe_standardize.md, read 2026-10-11) that produced typed TODOs exactly the way the prompt factory would. Nothing is presented as a Gru run that wasn't one -- the /claude confirmation is an explicit TODO for Bear.

Pipeline description for /snickerdoodle

Lectern is a weekly teaching-demand pipeline. It fetches job postings from three public, keyless ATS feeds (Greenhouse, Ashby, SmartRecruiters -- 18 watched boards), validates every record (title, URL, date, location, department; deduplicated on source + native ID), keeps the postings that are evidence of teaching interest using a tuned rule set with an audit loop, and produces a company demand report with every claim traceable to a raw posting in two clicks. The user is an independent AI-education consultant deciding which companies to approach about unposted work. Failure handling: untrusted runs freeze the closed-job logic, unlinked claims are hard stops, and the final report never ships without human approval.

TODO items from the audit

#	Type	Item
1	TODO: DATA SOURCE	Only one SmartRecruiters board (Canva) wired; 10 verified records, below the 25-record bar.
2	TODO: DEFINE	Trusted-run thresholds (>=90% of source-reported total, >=70% of previous trusted run) set from a single run; confirm after two more weekly runs.
3	TODO: DEV	New/gone native-ID diff runs by hand; script it.
4	TODO: APPROVE	Final demand report requires human approval every run (Phase Gate 4).
5	TODO: REPORT FIELD	"What's new this month" needs a human-written top-line takeaway label.
6	TODO: DEFINE	"Expected board count" per source is currently the previous run's snapshot; define the canonical board list in `sources.json` so drift is measured, not guessed. (Found in audit -- added.)
7	TODO: DEV	Agent output JSON (`logs/lectern-[DATE].json`) per the output contract is specified but not yet implemented. (Found in audit -- added.)

TODOs resolved before this submission

- **#6 (DEFINE):** RESOLVED -- the canonical board list lives in the existing sources.json (18 boards, committed in build-your-data-pipeline/); the recipe now names it as the drift reference instead of "last week's number."
- **#7 (DEV):** RESOLVED as specified-not-built -- the output contract in the recipe appendix defines the exact fields; implementation is a flagged DEV

task, not a vague intention.
- **Earlier resolutions carried in:** the collector, the rule set, the audit loop, and the two-click trace test were resolved in Assignments 3-4 and are marked RESOLVED in the recipe rather than re-audited.

TODOs still open and why

- **#1 (DATA SOURCE):** blocked on wiring more SmartRecruiters boards (needs board discovery work) or an explicit decision to run a two-source recipe.
- **#2 (DEFINE):** blocked on time -- needs two more weekly runs of data.
- **#3 (DEV):** blocked on implementation -- the diff logic is specified, the script isn't written.
- **#4 (APPROVE) / #5 (REPORT FIELD):** open by design -- these are per-run human judgments, not pre-closable items.

Gru /claude confirmation

TODO for Bear: paste the recipe into Gru, run /snickerdoodle with the pipeline description above, and run /claude once TODOs #1-#3 are closed. No confirmation is claimed here -- the recipe is audit-complete and rules-conformant to manual inspection, pending Gru's own verdict.